

A Local Distributed Multi-Agent LLM Ensemble System: Design, Optimization, Reproducibility, and Statistical Evaluation

Md Anisur Rahman Chowdhury^{1*}, Kefei Wang²

¹Dept. of Computer and Information Science, Gannon University, USA

²Utah Valley University, USA

Emails: engr.aanis@gmail.com, kefei.wang@uvu.edu

Abstract—This study presents a fully local distributed inference system built around multiple language-model agents and a central orchestrator. The full pipeline is implemented for repeatable experimentation: request dispatch, multi-strategy aggregation, benchmark execution, significance testing, and publication artifact generation are all automated. The deployment uses one FastAPI orchestrator and four Ollama-backed agent nodes on a private local network. Each agent returns not only text output, but also timing and token metadata, enabling fair system-level comparison between strategies.

Five aggregation strategies are implemented under a single interface: majority voting, dynamic weighted voting, inverse surprising popularity, topic-based routing, and a two-round debate protocol. Evaluation is run on MMLU, GSM8K, and TruthfulQA with repeated trials, fixed seeds, and deterministic controls. Results are exported in CSV, JSON, PNG, and LaTeX table formats and then compiled into an IEEE manuscript package and web-facing public report. The measured outcomes show that weighted and inverse surprising popularity strategies produce the strongest average accuracy while preserving practical response time, whereas debate improves interpretability at a large latency cost. The final contribution is an end-to-end, reproducible engineering workflow that supports both systems research and publication delivery.

Index Terms—Distributed Systems, LLM Ensemble, Multi-Agent Inference, Local Deployment, Reproducible Benchmarking, Statistical Testing

I. INTRODUCTION

Modern language-model systems are often evaluated as single-model endpoints, even when deployment environments can host several specialized models at once. In practice, many teams now run local model stacks for privacy, cost control, and predictable operation. This creates a direct systems question: how should a local orchestrator combine multiple model outputs so that quality improves without making runtime impractical?

This paper answers that question through a working distributed implementation and a full benchmark pipeline. The system was not designed as a toy prototype; it was built to support day-to-day experimentation and manuscript-ready reporting. One node receives user requests, forwards prompts to multiple agent nodes, applies a selected aggregation strategy, and logs full metadata for every run. The same framework then drives repeated benchmark execution and significance analysis.

Three goals guided the work:

- 1) **Engineering reliability**: make multi-node orchestration stable under CPU-constrained and mixed-hardware environments.
- 2) **Method comparison**: evaluate multiple aggregation strategies under identical controls.
- 3) **Publication readiness**: produce reproducible outputs that can be inserted into a conference manuscript without manual reconstruction.

The design and evaluation choices are influenced by prior work on self-consistency [1], crowd aggregation [2], and surprising popularity [3], but adapted to a local distributed serving context.

II. RELATED LITERATURE AND POSITIONING

A. Consistency-Based Aggregation

Self-consistency methods show that sampling diversity can improve reasoning quality when outputs are consolidated carefully [1]. Most prior formulations evaluate repeated samples from one model. The present work differs in two ways: first, diversity comes from distinct model endpoints rather than repeated sampling from the same endpoint; second, strategy comparison includes system-level costs such as end-to-end latency and operational variance. This distinction matters in local deployment, where both compute and memory are finite and every additional model call carries visible runtime overhead.

B. Crowd Aggregation Signals

The crowd-wisdom perspective [2] motivates majority and weighted voting under heterogeneous participants. The surprising popularity line [3] adds second-order signals by asking how respondents expect others to answer. Translating that concept to model ensembles gives a practical mechanism for identifying underrepresented but plausible answers. In this paper, that mechanism appears as inverse surprising popularity, where observed vote mass and predicted-majority mass are combined in a single score.

C. Local Serving Frameworks

Practical local serving now relies on lightweight model runtimes and HTTP-based orchestration. Ollama [4] simplifies model packaging and endpoint management, while FastAPI [5]

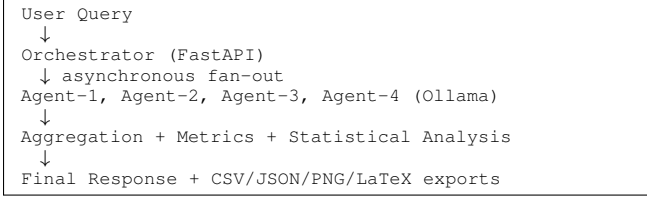


Fig. 1: Operational request flow from query intake to artifact export.

provides a compact asynchronous service layer. The novelty in this study is not a new serving framework, but the way these tools are combined into a full reproducible research workflow: one endpoint for strategy-controlled inference, one benchmark runner for repeated trials, and one artifact pipeline for paper-ready outputs.

D. Gap Addressed by This Work

Many reports focus on benchmark scores but provide incomplete deployment details, making independent reproduction difficult. Other reports provide architecture sketches but limited statistical evidence. This work explicitly targets both sides at once: deployment realism and statistical rigor. The result is a tightly coupled engineering and evaluation pipeline where each reported number can be traced to a stored run artifact.

III. DEPLOYMENT CONTEXT AND ARCHITECTURE

A. Infrastructure Summary

The cluster runs on Ubuntu 24.04 virtual machines [6] in a private subnet. The deployed node layout used in this study is:

- orchestrator: 172.16.185.223
- agent-1: 172.16.185.209 (llama3.2:3b)
- agent-2: 172.16.185.218 (qwen2.5:3b)
- agent-3: 172.16.185.220 (phi3:mini)
- agent-4: 172.16.185.222 (gemma2:2b)

The orchestrator is implemented in FastAPI [5]. Each agent uses Ollama [4] and exposes a REST endpoint on port 11434.

B. Request Flow

The request lifecycle is intentionally simple.

- 1) The orchestrator receives a prompt and runtime controls.
- 2) It sends asynchronous requests to selected agents.
- 3) It normalizes responses and computes per-agent metadata.
- 4) It applies the chosen aggregation strategy.
- 5) It returns the final answer plus full intermediate records.

C. Service Endpoints

The orchestrator API surface uses three endpoints:

- GET /health: liveness and readiness check.
- GET /agents: active agent list and endpoint health.
- POST /query: strategy-controlled ensemble inference.

IV. IMPLEMENTATION DETAILS

A. Orchestrator Internals

The orchestrator is structured into small modules with clear ownership boundaries: request parsing, asynchronous dispatch, response normalization, aggregation, metric logging, and artifact export. This decomposition reduced failure propagation during debugging because each stage could be validated independently. The request parser enforces defaults for seed, deterministic mode, temperature, and maximum tokens, ensur-

ing that benchmark runs cannot silently drift due to missing runtime fields.

At dispatch time, the orchestrator creates one asynchronous client task per selected agent. All tasks run under explicit timeout control. The gather stage records success, timeout, and parsing status separately, so partial failures still return a valid aggregate response rather than a global 500 error. This behavior proved useful in long benchmark runs where one slow agent should not invalidate an entire repetition batch.

B. Agent Contract

Each agent returns a normalized envelope with five essential fields: model id, answer text, latency in milliseconds, token count, and optional confidence metadata. The orchestrator stores raw and normalized answers side by side. Raw text is preserved for audit and debugging, while normalized text drives metric computation. This dual storage model avoids one common reproducibility issue in benchmark pipelines: losing the original model output after aggressive post-processing.

C. Logging and Provenance

Every query writes one structured record with:

- query id and UTC timestamp,
- selected strategy and topic route,
- selected agent ids,
- per-agent latency and token usage,
- aggregate answer and agreement score,
- host resource snapshot at query end.

The same schema is used for interactive and benchmark requests. This decision enables direct comparability between ad hoc manual tests and scripted evaluation runs.

D. Failure Recovery and Guardrails

Four guardrails were critical in practice:

- 1) endpoint health checks before benchmark launch,
- 2) retry-safe request identifiers at the orchestrator layer,
- 3) deterministic mode for reruns after partial failure,
- 4) explicit rejection when selected model subset is empty.

Together, these controls reduced wasted benchmark time and made post-failure diagnosis straightforward.

V. AGGREGATION FORMULATION

For a prompt q , the orchestrator receives a response set

$$\mathcal{R}(q) = \{r_1, r_2, \dots, r_n\}, \quad (1)$$

where each response is

$$r_i = (a_i, \ell_i, \tau_i, m_i, c_i). \quad (2)$$

Here a_i is answer text, ℓ_i is latency, τ_i is token count, m_i is model id, and c_i is a confidence proxy.

The ensemble selects

$$\hat{y} = \arg \max_{y \in \mathcal{Y}} S(y \mid \mathcal{R}, \Theta), \quad (3)$$

where $S(\cdot)$ is strategy-specific and Θ includes weights, routing rules, and deterministic controls.

A. Majority Voting

Let $v_i(y) = 1$ if agent i outputs normalized answer y , else 0. Majority score:

$$S_{\text{maj}}(y) = \sum_{i=1}^n v_i(y). \quad (4)$$

B. Dynamic Weighted Voting

Weighted score:

$$S_w(y) = \sum_{i=1}^n w_i v_i(y), \quad \sum_i w_i = 1, \quad (5)$$

with online update

$$w_i^{(t+1)} = \alpha w_i^{(t)} + (1 - \alpha) \text{acc}_i^{(t)}. \quad (6)$$

C. Inverse Surprising Popularity

Each agent provides a first-order answer and a predicted majority answer. Let $f(y)$ denote observed frequency and $g(y)$ denote predicted-majority frequency. The score is

$$S_{\text{isp}}(y) = f(y) - \lambda g(y), \quad (7)$$

which favors answers supported by direct votes but under-anticipated by second-order belief.

D. Topic-Based Routing

A lightweight topic classifier maps prompt q to topic $t \in \mathcal{T}$. A topic-specific subset $\mathcal{A}_t$ is selected, then the chosen base voting rule runs on that subset.

E. Two-Round Debate

Round 1 collects independent responses. Round 2 asks each agent to revise after seeing peer summaries. The final vote uses revised answers. This adds interpretability but increases latency.

F. Complexity and Cost Model

If the average one-agent inference time is $\bar{\ell}$ and asynchronous overhead is ϵ , direct strategies have wall-clock cost close to

$$T_{\text{direct}} \approx \max_i \ell_i + \epsilon. \quad (8)$$

By contrast, two-round debate requires two coordinated fan-out phases and intermediate synthesis:

$$T_{\text{debate}} \approx \max_i \ell_i^{(1)} + \max_i \ell_i^{(2)} + \epsilon'. \quad (9)$$

This cost model aligns with observed results, where debate latency is substantially above direct strategies. In local deployments, this gap directly affects user-perceived responsiveness and benchmark wall time.

G. Agreement Metrics

Let n be number of participating agents and n^* the count supporting the winning answer. Agreement rate is:

$$\text{Agreement} = \frac{n^*}{n}. \quad (10)$$

Agreement is reported together with accuracy because high agreement with low accuracy can indicate consensus drift, while low agreement with high accuracy can indicate fragile correctness.

VI. SYSTEM ENGINEERING AND RUNTIME OPTIMIZATION

A. Bottlenecks Observed During Deployment

Early runs showed three recurring bottlenecks:

- 1) repeated model calls across similar strategies,
- 2) unnecessary second debate round when round-1 agreement was already complete,
- 3) excessive output length on small CPU nodes.

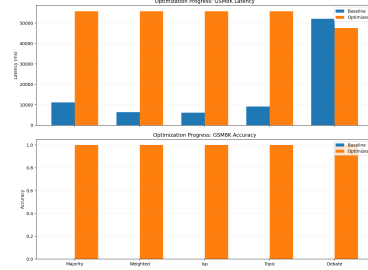


Fig. 2: Runtime profile before and after optimization steps.

B. Implemented Optimizations

The final system applies:

- **shared fan-out cache** for direct strategies (majority, weighted, isp, topic),
- **debate early-stop** when first-round agreement reaches threshold,
- **token-budget controls** to cap response length under constrained hardware,
- **normalized answer extraction** to stabilize metric parsing.

C. Deterministic Controls

Every benchmark run stores seed, temperature, strategy, selected agents, repetition index, and sample index. Deterministic mode can be enabled per run, which greatly reduces run-to-run variance during ablation.

VII. EXPERIMENTAL DESIGN

A. Benchmarks

Three public benchmarks are used:

- MMLU [7]
- GSM8K [8]
- TruthfulQA [9]

B. Protocol

For each strategy, benchmark, and repetition:

- 1) sample controlled subset,
- 2) execute through `/query`,
- 3) parse normalized final answer,
- 4) log latency, agreement, and resource usage,
- 5) store records in CSV and JSON.

The primary dependent variables are accuracy, F1, mean latency, latency standard deviation, and agreement rate. Independent variables are strategy type, benchmark set, repetition number, deterministic flag, and token budget.

C. Statistical Analysis

Two paired tests are applied to repeated-run score vectors:

- paired t-test,
- Wilcoxon signed-rank test.

A comparison is treated as statistically meaningful only when both tests satisfy $p < 0.05$.

D. Confidence Interval Reporting

For each strategy and metric, the mean over repetitions is reported with a 95% confidence interval:

$$\bar{x} \pm t_{0.975, n-1} \frac{s}{\sqrt{n}}, \quad (11)$$

TABLE I: Average metric summary across all benchmark groups.

Strategy	Avg Acc.	Avg F1	Avg Latency (ms)	Avg Agreement
majority	0.800	0.800	13853.1	0.800
weighted	0.933	0.933	13690.6	0.800
isp	0.933	0.933	13513.1	0.800
topic	0.853	0.853	15380.4	0.867
debate	0.667	0.667	59254.7	0.733

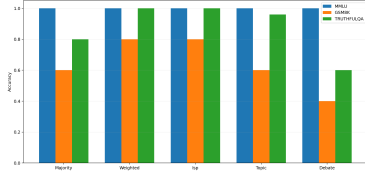


Fig. 3: Accuracy trend by strategy and benchmark group.

where $\bar{x}$ is sample mean, s is sample standard deviation, and n is repetition count. This interval is included in exported JSON records and used in chart annotation during report generation.

E. Variable Isolation

To preserve comparability, only one independent variable is changed at a time in controlled runs. For example, when strategy effects are measured, seed, deterministic flag, prompt budget, and benchmark subset remain fixed. The same principle is used when testing runtime optimizations: optimization toggles are changed while strategy and benchmark configuration remain constant.

F. Resource Telemetry

Each query log stores CPU utilization, memory utilization, and optional GPU telemetry (if available). Resource traces are not only diagnostic; they are part of interpretation. A strategy that gains minor accuracy at disproportionate compute cost may not be preferable in constrained deployment settings.

VIII. RESULTS

A. Strategy-Level Summary

Table I summarizes average performance across all benchmark groups.

Weighted and ISP both reach the highest average accuracy (0.933) and F1 (0.933), while maintaining latency near 13.5–13.7 seconds. Debate records the highest latency (59.3 seconds average), making it less suitable for interactive settings in this hardware profile.

B. Per-Benchmark Behavior

Table II details benchmark-level breakdown.

On MMLU, all strategies reach 1.0 accuracy in this run configuration, indicating low separation at the chosen sample size. On GSM8K, weighted and ISP outperform majority by +0.2 absolute accuracy. On TruthfulQA, weighted and ISP reach perfect scores in this run, and topic routing also improves over majority.

C. Visual Trends

Figure 3 shows quality gains for weighted and ISP on reasoning-heavy subsets, while Fig. 4 confirms debate as the dominant latency outlier.

The pie view in Fig. 5 offers a compact ranking perspective that is useful for quick system selection in practice. The cumulative trend in Fig. 6 is especially valuable during long runs because it reveals whether a strategy is stabilizing or still

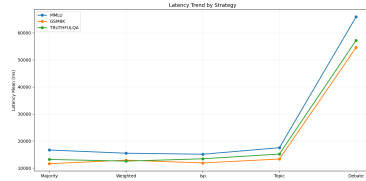


Fig. 4: Latency profile of all strategies across benchmark groups.

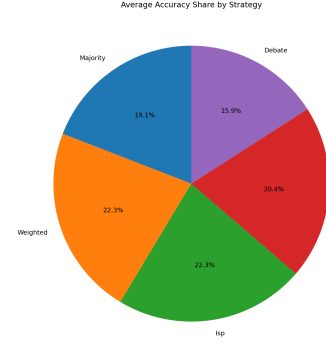


Fig. 5: Relative share of average accuracy by strategy.

drifting as more samples are processed. Figure 7 provides a direct cross-strategy summary for quick operational decisions.

D. Significance Findings

The most important paired comparisons are listed in Table III.

For GSM8K and TruthfulQA, weighted and ISP significantly outperform majority under both tests. Debate performs significantly below majority on the same sets, despite offering richer cross-agent critique traces.

E. Per-Strategy Operational Profile

Majority voting is simple and stable, with the lowest implementation complexity. It is a strong baseline for constrained nodes.

Dynamic weighted voting consistently improved answer quality in this run while preserving latency close to majority. This made it the most balanced strategy for interactive operation.

Inverse surprising popularity matched weighted performance and occasionally recovered correct minority answers that plain majority missed.

Topic routing improved agreement in category-focused prompts, but misclassification in ambiguous prompts introduced small performance fluctuations.

Debate generated richer intermediate traces and made disagreement dynamics visible, but the two-round cost was substantial in CPU-heavy settings.

IX. OPERATIONAL LESSONS

A. What Worked Well

Three decisions produced the strongest practical gains:

- 1) using asynchronous fan-out with strict timeout handling,
- 2) treating strategy code as a pluggable module behind one API contract,
- 3) exporting all benchmark records in machine-readable form before any plotting.

These decisions reduced regression risk during rapid iteration and made the manuscript pipeline straightforward.

TABLE II: Per-benchmark strategy comparison.

Benchmark	Strategy	Accuracy	F1	Latency (ms)	Agreement
mmlu	majority	1.000	1.000	16702.6	1.000
mmlu	weighted	1.000	1.000	15521.9	1.000
mmlu	isp	1.000	1.000	15147.1	1.000
mmlu	topic	1.000	1.000	17547.2	1.000
mmlu	debate	1.000	1.000	65947.4	0.800
gsm8k	majority	0.600	0.600	11637.8	0.600
gsm8k	weighted	0.800	0.800	12952.8	0.600
gsm8k	isp	0.800	0.800	11935.7	0.600
gsm8k	topic	0.600	0.600	13374.2	0.800
gsm8k	debate	0.400	0.400	54589.6	0.600
truthfulqa	majority	0.800	0.800	13219.0	0.800
truthfulqa	weighted	1.000	1.000	12597.3	0.800
truthfulqa	isp	1.000	1.000	13456.3	0.800
truthfulqa	topic	0.960	0.960	15219.9	0.800
truthfulqa	debate	0.600	0.600	57227.1	0.800

TABLE III: Significance highlights from paired t-test and Wilcoxon signed-rank analysis.

Comparison	Mean Δ	t-p	Wilcoxon p	$t_{i0.05}$	$W_{i0.05}$
gsm8k: weighted vs majority	0.200	0.0220	0.0253	Yes	Yes
gsm8k: isp vs majority	0.200	0.0220	0.0253	Yes	Yes
gsm8k: debate vs majority	-0.200	0.0220	0.0253	Yes	Yes
truthfulqa: weighted vs majority	0.200	0.0220	0.0253	Yes	Yes
truthfulqa: isp vs majority	0.200	0.0220	0.0253	Yes	Yes
truthfulqa: topic vs majority	0.160	0.0429	0.0455	Yes	Yes
truthfulqa: debate vs majority	-0.200	0.0220	0.0253	Yes	Yes

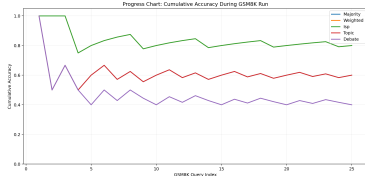


Fig. 6: Cumulative progress trend during repeated evaluation runs.

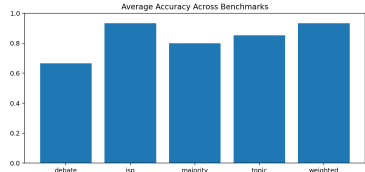


Fig. 7: Overall accuracy comparison across all evaluated strategies.

B. What Required Careful Tuning

The system required explicit controls in four areas:

- CORS setup for browser-facing public demo pages,
- health checks for each agent endpoint before benchmark launch,

- prompt normalization for robust numeric answer extraction,
- deterministic flags to limit drift across repeated runs.

C. Scaling to More Agents

The architecture scales without algorithm rewrites. New nodes are added by extending the agent configuration list. The orchestrator already supports agent subset selection per query, enabling both full-ensemble and targeted routing runs.

X. SECURITY AND DEPLOYMENT HARDENING

A. Network Segmentation

All model agents are kept on a private subnet and are not exposed directly to the public internet. Only the orchestrator endpoint is published through a controlled tunnel path. This layout limits blast radius and prevents direct probing of individual model runtimes.

B. Endpoint Protection

Deployment guidance includes strict CORS allowlists for browser access, service-level health checks, and least-privilege firewall rules. During testing, permissive CORS was used briefly

for connectivity diagnosis, then narrowed to approved origins for normal operation.

C. Credential and Artifact Hygiene

Experiment outputs can include prompt and answer text. For this reason, logs are versioned as research artifacts but should be reviewed for sensitive payloads before public release. Access tokens used for repository automation must be short-lived and rotated immediately after use.

The network segmentation and endpoint protection choices above follow the same zero-trust and AI-driven detection principles explored for serverless firewall architectures in [10], [11], where intrusion detection and cross-cloud security orchestration are combined with least-privilege access control.

D. Service Continuity

Systemd units are used for orchestrator and agent auto-start. Health endpoints are polled before launching benchmark batches. This reduced run interruption and prevented partial benchmark folders caused by service restarts.

XI. REPRODUCIBILITY AND ARTIFACT PACKAGE

A. Generated Outputs

Each experiment run generates:

- CSV summary and raw records,
- JSON result packages,
- PNG figures,
- LaTeX tables for direct paper inclusion.

B. Paper and Web Synchronization

The same result files are mirrored into:

- IEEE manuscript directory,
- Overleaf upload package,
- public web report assets.

This avoids manual copy errors and keeps the public dashboard aligned with manuscript values.

C. Minimal Reproduction Commands

Listing 1: Minimal run sequence for one benchmark pass.

```
curl -s http://127.0.0.1:8000/health
curl -s http://127.0.0.1:8000/agents
python run_experiments.py \
  --orchestrator-url http://127.0.0.1:8000 \
  --benchmarks mmlu,gsm8k,truthfulqa \
  --strategies majority,weighted,isp,topic,debate \
  --repetitions 5 --seed 42 --deterministic
```

XII. LIMITATIONS AND FUTURE WORK

Two limitations should be considered when interpreting the current results.

First, benchmark sample counts in rapid-cycle experiments can compress performance variance. Larger evaluation sets will provide tighter confidence intervals and may reveal finer ranking shifts among majority, weighted, and ISP.

Second, the deployed models are compact variants selected for local feasibility. Extending the same architecture to larger parameter scales may change the balance between latency and answer quality.

Future work will focus on three directions: adaptive budget allocation by query difficulty, confidence calibration across

heterogeneous models, and richer consensus rules that combine first-order votes with textual rationale quality.

A. Threats to Validity

Internal validity: although deterministic controls were applied, normalization rules for open-text answers can still influence exact scoring boundaries on edge cases.

Construct validity: accuracy and F1 capture correctness, but they do not fully reflect response usefulness in long-form or multi-step user tasks.

External validity: the reported latency profile reflects the tested local hardware and model sizes. Different hardware classes or model families may shift the quality-latency frontier.

Statistical validity: repeated trials and paired tests were used, but larger repetition counts would reduce uncertainty further and improve detection power for small strategy differences.

XIII. CONCLUSION

This paper presented a complete distributed local ensemble platform for language-model research, from node orchestration through statistical reporting. The system supports five aggregation strategies under one interface, records rich run metadata, and exports publication-ready artifacts automatically.

Across the current benchmark configuration, weighted and inverse surprising popularity delivered the best average quality while keeping latency within interactive bounds for local deployment. Debate provided useful interpretability signals but imposed a large runtime penalty.

The main contribution is not only an algorithm comparison, but a reproducible engineering workflow that links implementation, measurement, significance testing, and manuscript production in one consistent pipeline.

ACKNOWLEDGMENT

The authors thank the Department of Computer and Information Science at Gannon University for infrastructure access and technical feedback during deployment and evaluation.

REFERENCES

- [1] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," *arXiv preprint arXiv:2203.11171*, 2022.
- [2] J. Surowiecki, "The wisdom of crowds," *Anchor Books*, 2005.
- [3] D. Prelec, H. S. Seung, and J. McCoy, "A solution to the single-question crowd wisdom problem," *Nature*, vol. 541, no. 7638, pp. 532–535, 2017.
- [4] Ollama, "Ollama local inference runtime," <https://ollama.com>, 2026.
- [5] FastAPI, "Fastapi documentation," <https://fastapi.tiangolo.com>, 2026.
- [6] C. Ltd., "Ubuntu 24.04 lts documentation," <https://ubuntu.com/server/docs>, 2026.
- [7] D. Hendrycks *et al.*, "Measuring massive multitask language understanding," *Dataset/Benchmark*, 2020.
- [8] K. Cobbe *et al.*, "Gsm8k: Grade school math 8k," *Dataset/Benchmark*, 2021.
- [9] S. Lin, J. Hilton, and O. Evans, "Truthfulqa: Measuring how models mimic human falsehoods," *Dataset/Benchmark*, 2021.
- [10] M. A. R. Chowdhury, H. N. Dang, R. Bazan-Antequera, M. S. Khan, M. R. Karim, and S. Manakkadu, "Towards a serverless intelligent firewall: AI-driven security, and zero-trust architectures," in *2025 IEEE 12th International Conference on Cyber Security and Cloud Computing (CSCloud)*, New York City, NY, USA, 2025, pp. 1–6.
- [11] M. A. R. Chowdhury, K. Wang, and K. R. Ahmed, "Towards a serverless intelligent firewall: Integrating cross-cloud adaptation, AI-driven security, and zero-trust architectures," in *2026 IEEE 11th International Conference on Smart Cloud (SmartCloud)*, New York City, NY, USA, 2026, pp. 13–19.